

Système de Gestion Scolaire

Application Web Spring Boot

Gestion des élèves, filières et cours

Développement d'une application web complète
pour la gestion administrative d'un établissement scolaire
avec Spring Boot et Thymeleaf

Informations générales

- **Nom du projet** : Système de Gestion Scolaire
- **Cadre** : Projet de développement Spring Boot - Thymeleaf
- **Domaine** : Gestion scolaire et administration éducative
- **Réalisé par** : ARIB Aymane
- **Technologies principales** : Spring Boot, Spring Data JPA, Thymeleaf, Bootstrap 5, H2 Database
- **Année universitaire** : 2025 – 2026

Table des matières

1	INTRODUCTION ET CONTEXTE	3
2	ARCHITECTURE TECHNIQUE	3
2.1	Stack Technologique	3
2.2	Structure du Projet	4
2.3	Analyse de l'Architecture en Couches	4
2.3.1	Couche Présentation (Controllers)	5
2.3.2	Couche Métier (Services)	5
2.3.3	Couche Persistance (Repositories)	5
2.3.4	Couche Vue (Templates Thymeleaf)	6
2.4	Architecture Logique en 4 Couches	6
3	MODÈLE DE DONNÉES	7
3.1	Entités Implémentées	7
3.1.1	Entité Eleve (Étudiant)	7
3.1.2	Entité Filière	8
3.1.3	Entité Cours	8
3.1.4	Entité DossierAdministratif	9
3.2	Diagramme Conceptuel de la Base de Données	10
4	FONCTIONNALITÉS IMPLÉMENTÉES	10
4.1	CRUD Complet par Entité	11
4.1.1	Gestion des Élèves	11
4.1.2	Gestion des Filières	11
4.1.3	Gestion des Cours	11
4.2	Fonctionnalités Spéciales	11
4.2.1	Génération Automatique de Numéro d'Inscription	11
4.2.2	Création Automatique du Dossier Administratif	12
4.2.3	Interface Utilisateur Avancée	12
5	INTERFACES UTILISATEUR	12
5.1	Page d'Accueil	13
5.2	Gestion des Élèves	13
5.2.1	Liste des Élèves	13
5.2.2	Formulaire d'Ajout/Modification	14
5.2.3	Détails d'un Élève	14
5.3	Gestion des Filières	15
5.3.1	Liste des Filières	15
5.3.2	Formulaire de Filière	15
5.4	Gestion des Cours	15
5.4.1	Catalogue des Cours	15
5.4.2	Formulaire de Cours	16

6	ASPECTS TECHNIQUES AVANCÉS	16
6.1	Gestion des Transactions	16
6.2	Sécurité et Validation	17
6.3	Optimisation des Performances	17
6.3.1	Lazy Loading	17
7	DIFFICULTÉS RENCONTRÉES ET SOLUTIONS	17
7.1	Problème : Gestion des Relations ManyToMany Bidirectionnelles	18
7.2	Problème : Génération Automatique du Dossier Administratif	18
7.3	Problème : Intégration Thymeleaf avec Bootstrap	18
8	CONCLUSION	19

1 INTRODUCTION ET CONTEXTE

Ce projet a été développé dans le cadre de l'apprentissage approfondi du framework **Spring Boot**, leader dans le développement d'applications Java entreprise. L'objectif principal était de concevoir et implémenter une application web complète de gestion scolaire, permettant d'administrer de manière centralisée les élèves, filières, cours et dossiers administratifs au sein d'un établissement éducatif.

Dans un contexte où la digitalisation des processus administratifs devient cruciale pour l'efficacité opérationnelle, ce système répond aux besoins fondamentaux d'une institution scolaire moderne. Il offre une plateforme intuitive et performante pour gérer les informations académiques, tout en démontrant les capacités du framework Spring Boot dans la construction d'applications web robustes et maintenables.

2 ARCHITECTURE TECHNIQUE

L'architecture du système repose sur les principes SOLID et suit le pattern MVC (Model-View-Controller), adapté à l'écosystème Spring Boot. Cette approche garantit une séparation claire des responsabilités, une maintenabilité optimale et une évolutivité à long terme.

2.1 Stack Technologique

Le projet utilise une stack technologique moderne et cohérente, articulée autour des composants suivants :

Composant	Description
Spring Boot 3.x	Framework principal pour la configuration automatique et le démarrage rapide
Spring Data JPA	Abstraction pour la persistance des données et le mapping objet-relationnel
Spring MVC	Architecture pour la gestion des requêtes HTTP et le routage
H2 Database	Base de données relationnelle en mémoire, idéale pour le développement
Thymeleaf	Moteur de templates côté serveur pour la génération des vues
Bootstrap 5.3	Framework CSS pour des interfaces utilisateur responsives et modernes
Lombok	Bibliothèque pour réduire le code boilerplate via des annotations
Maven	Outil de gestion des dépendances et d'automatisation de build

TABLE 1 – Stack technologique du projet

2.2 Structure du Projet

L'organisation du code source suit les conventions standards de Spring Boot, avec une séparation claire en couches :

Listing 1 – Structure du projet

```
1  src/main/java/org/example/demo/
2      Controllers/
3          CoursController.java
4          EleveController.java
5          FiliereController.java
6          HomeController.java
7      Entities/
8          Cours.java
9          DossierAdministratif.java
10         Eleve.java
11         Filiere.java
12     Repository/
13         CoursRepository.java
14         DossierAdministratifRepository.java
15         EleveRepository.java
16         FiliereRepository.java
17     Service/
18         CoursService.java
19         EleveService.java
20         FiliereService.java
21         NumeroInscriptionGenerator.java
22     DemoApplication.java
23
24  src/main/resources/
25      templates/
26          cours/
27              form.html
28              list.html
29          eleves/
30              details.html
31              form.html
32              list.html
33          filieres/
34              details.html
35              form.html
36              list.html
37          index.html
38          layout.html
39      static/
40          css/
41          js/
42      application.properties
43      data.sql (optionnel)
```

2.3 Analyse de l'Architecture en Couches

L'architecture suit rigoureusement le pattern en couches, avec une séparation nette des responsabilités :

2.3.1 Couche Présentation (Controllers)

Les contrôleurs gèrent les requêtes HTTP et orchestrent le flux entre la vue et le service :

Listing 2 – Exemple de contrôleur

```
1 @Controller
2 @RequestMapping("/eleves")
3 public class EleveController {
4
5     @Autowired
6     private EleveService eleveService;
7
8     @GetMapping
9     public String listEleves(Model model) {
10         model.addAttribute("eleves", eleveService.getAllEleves());
11         return "eleves/list";
12     }
13
14     @GetMapping("/{id}")
15     public String viewEleve(@PathVariable Long id, Model model) {
16         Eleve eleve = eleveService.getEleveById(id);
17         model.addAttribute("eleve", eleve);
18         return "eleves/details";
19     }
20 }
```

2.3.2 Couche Métier (Services)

Les services implémentent la logique métier et coordonnent les transactions :

Listing 3 – Exemple de service

```
1 @Service
2 @Transactional
3 public class EleveService {
4
5     @Autowired
6     private EleveRepository eleveRepository;
7
8     @Autowired
9     private NumeroInscriptionGenerator numeroGenerator;
10
11     public Eleve createEleve(Eleve eleve, Long filiereId, List<Long>
12         coursIds) {
13         // Logique m tier complexe
14         eleve.setDossierAdministratif(createDossier(eleve));
15         return eleveRepository.save(eleve);
16     }
17 }
```

2.3.3 Couche Persistance (Repositories)

Les repositories fournissent l'abstraction d'accès aux données via Spring Data JPA :

Listing 4 – Exemple de repository

```

1 @Repository
2 public interface EleveRepository extends JpaRepository<Eleve, Long> {
3
4     List<Eleve> findByFiliereId(Long filiereId);
5
6     @Query("SELECT e FROM Eleve e WHERE e.nom LIKE %:nom%")
7     List<Eleve> searchByNom(@Param("nom") String nom);
8 }

```

2.3.4 Couche Vue (Templates Thymeleaf)

Les templates génèrent le HTML dynamique avec intégration Bootstrap :

Listing 5 – Exemple de template Thymeleaf

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4     <title>Liste des lves </title>
5     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/
6         bootstrap.min.css" rel="stylesheet">
7 </head>
8 <body>
9     <div class="container mt-4">
10         <h1>Liste des lves </h1>
11         <table class="table table-striped">
12             <thead>
13                 <tr>
14                     <th>ID</th>
15                     <th>Nom</th>
16                     <th>Pr nom</th>
17                     <th>Actions</th>
18                 </tr>
19             </thead>
20             <tbody>
21                 <tr th:each="eleve : ${eleves}">
22                     <td th:text="${eleve.id}"></td>
23                     <td th:text="${eleve.nom}"></td>
24                     <td th:text="${eleve.prenom}"></td>
25                     <td>
26                         <a th:href="@{/eleves/{id}(id=${eleve.id})}"
27                             class="btn btn-info btn-sm">Voir</a>
28                     </td>
29                 </tr>
30             </tbody>
31         </table>
32     </div>
33 </body>
34 </html>

```

2.4 Architecture Logique en 4 Couches

Couche 1 : Présentation

- Technologies : Thymeleaf, Bootstrap 5, JavaScript
- Responsabilité : Interface utilisateur, navigation, affichage

— Pattern : Controller-Service

Couche 2 : Métier

— Technologies : Spring Services, Transactions

— Responsabilité : Règles métier, validation, coordination

— Pattern : Service Layer

Couche 3 : Persistance

— Technologies : Spring Data JPA, Hibernate

— Responsabilité : Accès aux données, ORM, requêtes

— Pattern : Repository

Couche 4 : Base de données

— Technologie : H2 Database (en mémoire)

— Avantages : Aucune installation, rapide, parfaite pour le développement

3 MODÈLE DE DONNÉES

Le modèle de données a été conçu pour répondre aux besoins d'un système de gestion scolaire moderne, avec des entités clairement définies et des relations appropriées.

3.1 Entités Implémentées

Quatre entités principales ont été développées avec JPA, chacune représentant un concept métier fondamental :

3.1.1 Entité Eleve (Étudiant)

Listing 6 – Classe Entité Eleve

```
1 @Getter @Setter
2 @AllArgsConstructor @NoArgsConstructor
3 @Entity
4 public class Eleve {
5
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    private String nom;
11    private String prenom;
12
13    @ManyToOne
14    @JoinColumn(name = "filiere_id")
15    private Filiere filiere;
16
17    @ManyToMany
18    @JoinTable(
19        name = "eleve_cours",
20        joinColumns = @JoinColumn(name = "eleve_id"),
21        inverseJoinColumns = @JoinColumn(name = "cours_id")
22    )
```



```

23     private List<Cours> cours = new ArrayList<>();
24
25     @OneToOne(mappedBy = "eleve", cascade = CascadeType.ALL,
26               orphanRemoval = true)
27     private DossierAdministratif dossierAdministratif;
28
29     public void setDossierAdministratif(DossierAdministratif d) {
30         this.dossierAdministratif = d;
31         if (d != null) d.setEleve(this);
32     }

```

Attributs :

- id : Identifiant unique (auto-généré)
- nom, prenom : Informations personnelles
- Relations : Filière (ManyToOne), Cours (ManyToMany), Dossier (OneToOne)

3.1.2 Entité Filière

Listing 7 – Classe Entité Filière

```

1  @Getter @Setter
2  @AllArgsConstructor @NoArgsConstructor
3  @Entity
4  public class Filiere {
5
6      @Id
7      @GeneratedValue(strategy = GenerationType.IDENTITY)
8      private Long id;
9
10     private String code;
11     private String nom;
12
13     @OneToMany(mappedBy = "filiere")
14     private List<Eleve> eleves = new ArrayList<>();
15
16     @OneToMany(mappedBy = "filiere")
17     private List<Cours> cours = new ArrayList<>();
18 }

```

3.1.3 Entité Cours

Listing 8 – Classe Entité Cours

```

1
2  @AllArgsConstructor @NoArgsConstructor
3  @Getter @Setter
4  @Entity
5  public class Cours {
6
7      @Id
8      @GeneratedValue(strategy = GenerationType.IDENTITY)
9      private Long id;
10
11     private String code;

```

```
12     private String intitule;  
13  
14     @ManyToOne  
15     @JoinColumn(name = "filiere_id")  
16     private Filiere filiere;  
17  
18     @ManyToMany(mappedBy = "cours")  
19     private List<Eleve> eleves = new ArrayList<>();  
20 }
```

3.1.4 Entité DossierAdministratif

Listing 9 – Classe Entité DossierAdministratif

```
1 @Getter @Setter  
2 @AllArgsConstructor @NoArgsConstructor  
3 @Entity  
4 public class DossierAdministratif {  
5  
6     @Id  
7     @GeneratedValue(strategy = GenerationType.IDENTITY)  
8     private Long id;  
9  
10    private String numeroInscription;  
11    private LocalDate dateCreation;  
12  
13    @OneToOne  
14    @JoinColumn(name = "eleve_id", unique = true)  
15    private Eleve eleve;  
16 }
```

3.2 Diagramme Conceptuel de la Base de Données

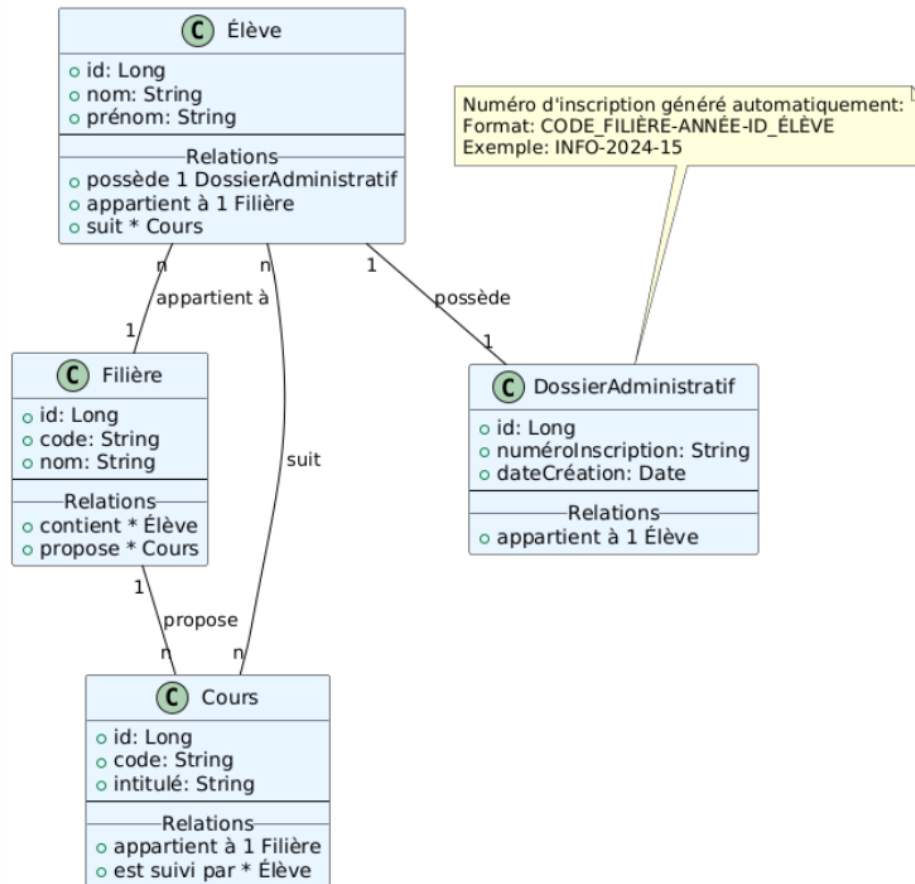


FIGURE 1 – Diagramme ER du système de gestion scolaire

Relations principales :

- Une **Filière** a plusieurs **Élèves** (OneToMany)
- Un **Élève** appartient à une **Filière** (ManyToOne)
- Un **Élève** suit plusieurs **Cours** (ManyToMany)
- Un **Cours** appartient à une **Filière** (ManyToOne)
- Un **Élève** a un **DossierAdministratif** (OneToOne)

4 FONCTIONNALITÉS IMPLÉMENTÉES

Le système propose un ensemble complet de fonctionnalités couvrant tous les aspects de la gestion scolaire, depuis l'inscription des élèves jusqu'à la gestion des programmes académiques.

4.1 CRUD Complet par Entité

4.1.1 Gestion des Élèves

Fonctionnalité	Description
Ajout d'élève	Formulaire avec sélection de filière et de cours multiples
Modification	Mise à jour des informations personnelles et académiques
Suppression	Suppression avec confirmation et gestion des dépendances
Consultation	Vue détaillée avec filière, cours et dossier administratif
Recherche	Recherche par nom, filière ou cours

TABLE 2 – Fonctionnalités de gestion des élèves

4.1.2 Gestion des Filières

- **Création** : Ajout de nouveaux programmes académiques avec code unique
- **Modification** : Mise à jour des informations descriptives
- **Suppression** : Gérée avec vérification des élèves associés
- **Listage** : Affichage avec statistiques (nombre d'élèves, cours)

4.1.3 Gestion des Cours

- **Ajout** : Création de nouvelles matières avec rattachement à une filière
- **Édition** : Modification des informations pédagogiques
- **Suppression** : Contrôlée pour éviter les incohérences
- **Catalogue** : Vue complète des cours disponibles par filière

4.2 Fonctionnalités Spéciales

4.2.1 Génération Automatique de Numéro d'Inscription

Une fonctionnalité avancée a été implémentée pour générer automatiquement des numéros d'inscription uniques :

Listing 10 – Générateur de numéro d'inscription

```

1 @Component
2 public class NumeroInscriptionGenerator {
3
4     public String generate(Filiere filiere, Long eleveId) {
5         String code = (filiere != null && filiere.getCode() != null && !
6             filiere.getCode().isBlank()
7             ? filiere.getCode().toUpperCase()
8             : "NA");
9
10        int annee = Year.now().getValue();
11        return code + "-" + annee + "-" + eleveId;
12    }
13 }

```

Format : CODE_FILIERE-ANNEE-ID_ELEVE (ex : INFO-2025-42)

4.2.2 Création Automatique du Dossier Administratif

À chaque création d'élève, un dossier administratif est automatiquement généré :

Listing 11 – Création automatique du dossier

```
1 @Transactional
2 public Eleve createEleve(Eleve eleve, Long filiereId, List<Long>
   coursIds) {
3     // ... initialisation de l' lve
4
5     // Cr ation automatique du dossier
6     DossierAdministratif dossier = new DossierAdministratif();
7     dossier.setDateCreation(LocalDate.now());
8     dossier.setNumeroInscription(
9         numeroGenerator.generate(eleve.getFiliere(), eleve.getId())
10    );
11    dossier.setEleve(eleve);
12    eleve.setDossierAdministratif(dossier);
13
14    return eleveRepository.save(eleve);
15 }
```

4.2.3 Interface Utilisateur Avancée

- **Design responsive** : Adaptation à tous les écrans (mobile, tablette, desktop)
- **Navigation intuitive** : Menu fixe avec highlights visuels
- **Recherche en temps réel** : Filtrage dynamique des listes
- **Feedback utilisateur** : Messages de confirmation, erreurs, succès
- **Compatibilité** : Support multi-navigateurs (Chrome, Firefox, Safari)

5 INTERFACES UTILISATEUR

Les interfaces ont été conçues avec Bootstrap 5 pour offrir une expérience utilisateur moderne, intuitive et professionnelle.

5.1 Page d'Accueil



FIGURE 2 – Tableau de bord principal

Caractéristiques :

- Vue d'ensemble avec statistiques (nombre d'élèves, filières, cours)
- Cartes de navigation vers les principales sections
- Design moderne avec gradient et icônes FontAwesome de navigation fixe pour un accès rapide

5.2 Gestion des Élèves

5.2.1 Liste des Élèves

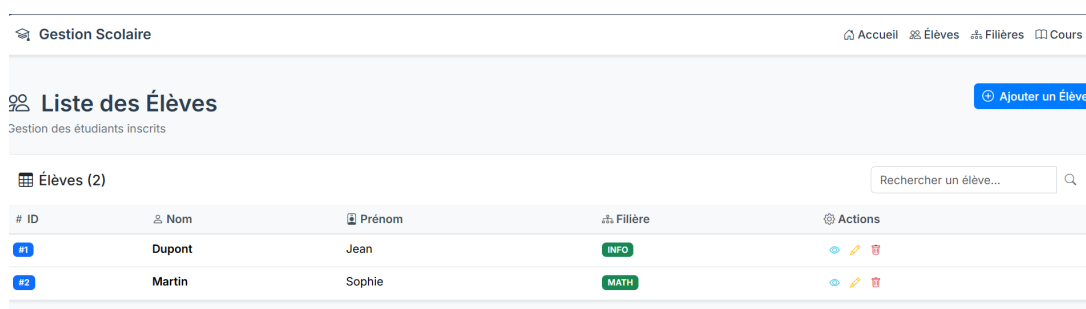


FIGURE 3 – Interface de liste des élèves

Fonctionnalités :

- Tableau paginé avec tri sur toutes les colonnes
- Barre de recherche en temps réel
- Actions rapides (Voir, Éditer, Supprimer)
- Badges pour indiquer la filière

5.2.2 Formulaire d’Ajout/Modification

The screenshot shows a web form titled "Ajouter un Élève". It contains the following fields and elements:

- Nom**: A text input field.
- Prénom**: A text input field.
- Filière**: A dropdown menu with the selected option "-- Aucune --".
- Cours**: A section with three checkboxes:
 - ☐ Programmation Java
 - ☐ Algorithmique
 - ☐ Calcul Différentiel
- Buttons**: Two buttons at the bottom, "Enregistrer" (green) and "Retour" (grey).

FIGURE 4 – Formulaire de gestion d’élève

Éléments :

- Champs de saisie avec validation
- Sélection déroulante pour la filière
- Sélection multiple pour les cours
- Boutons d’action clairement identifiés

5.2.3 Détails d’un Élève

The screenshot shows a web page titled "Détails élève". It features a navigation bar at the top with "Gestion Scolaire" and links for "Accueil", "Élèves", "Filières", and "Cours". The main content area displays the following information:

- Détails élève**:
 - Nom** : Dupont
 - Prénom** : Jean
 - Filière** : Informatique
- Dossier administratif**:
 - Numéro** : INFO-2026-1
- Cours suivis**:
 - Programmation Java
 - Algorithmique

A "Retour" button is located at the bottom of the page.

FIGURE 5 – Fiche détaillée d’un élève

Informations affichées :

- Données personnelles
- Filière et cours suivis
- Dossier administratif avec numéro d’inscription vers les entités associées

5.3 Gestion des Filières

5.3.1 Liste des Filières

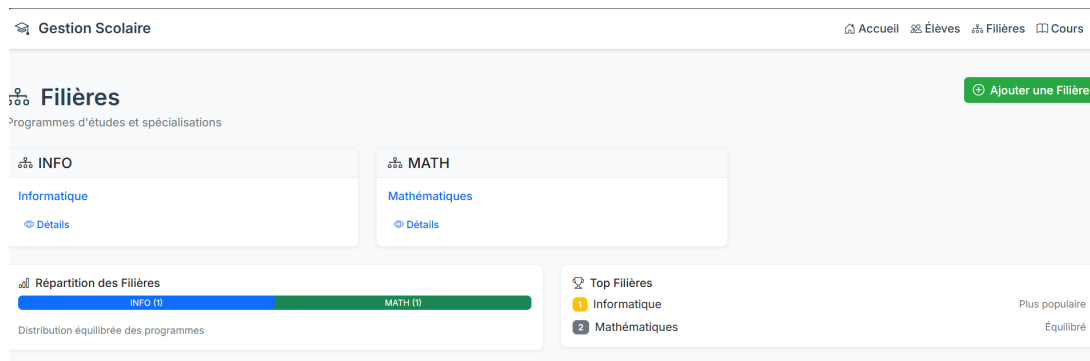


FIGURE 6 – Interface de gestion des filières

5.3.2 Formulaire de Filière

Gestion Scolaire Accueil Éléves Filières Cours

Ajouter une filière

Code

Nom

Enregistrer Retour

FIGURE 7 – Formulaire de création/modification de filière

5.4 Gestion des Cours

5.4.1 Catalogue des Cours

Gestion Scolaire Accueil Éléves Filières Cours

Cours

Catalogue des matières enseignées

Ajouter un Cours

Rechercher un cours...

#	Code	Intitulé	Filière	Éléves inscrits	Actions
	JAVA101	Programmation Java	INFO	0	
	ALGO101	Algorithmique	INFO	0	
	CALC201	Calcul Différentiel	MATH	0	

Cours actifs: 1

Filières couvertes: 2

Inscriptions totales: 0

FIGURE 8 – Liste des cours disponibles

5.4.2 Formulaire de Cours

FIGURE 9 – Interface de gestion des cours

6 ASPECTS TECHNIQUES AVANCÉS

Le projet intègre plusieurs aspects techniques avancés qui démontrent une maîtrise approfondie du framework Spring Boot et des bonnes pratiques de développement.

6.1 Gestion des Transactions

La gestion transactionnelle est cruciale pour maintenir l'intégrité des données. Spring Boot simplifie cette gestion via l'annotation `@Transactional` :

Listing 12 – Gestion transactionnelle avancée

```

1  @Service
2  @Transactional
3  public class EleveService {
4
5      @Autowired
6      private EleveRepository eleveRepository;
7
8      @Autowired
9      private FiliereRepository filiereRepository;
10
11     @Transactional(propagation = Propagation.REQUIRED,
12                     rollbackFor = Exception.class)
13     public Eleve createEleve(String nom, String prenom,
14                             Long filiereId, List<Long> coursIds) {
15
16         // Rcupration de la filiere
17         Filiere filiere = filiereRepository.findById(filiereId)
18             .orElseThrow(() -> new ResourceNotFoundException(
19                 "Fili re non trouv e avec l'ID: " + filiereId));
20
21         // Cr ation de l' lve
22         Eleve eleve = new Eleve();
23         eleve.setNom(nom);
24         eleve.setPrenom(prenom);
25         eleve.setFiliere(filiere);
26
27         // Sauvegarde initiale pour obtenir l'ID
28         eleve = eleveRepository.save(eleve);
29

```

```
30 // Cr ation automatique du dossier administratif
31 DossierAdministratif dossier = new DossierAdministratif();
32 dossier.setDateCreation(LocalDate.now());
33 dossier.setNumeroInscription(
34     numeroGenerator.generate(filiere, eleve.getId()));
35 eleve.setDossierAdministratif(dossier);
36
37 // Sauvegarde finale avec toutes les d pendances
38 return eleveRepository.save(eleve);
39 }
40 }
```

Avantages :

- Atomicité des opérations
- Rollback automatique en cas d'erreur
- Cohérence des données garantie

6.2 Sécurité et Validation

La validation des données est essentielle pour la robustesse de l'application :

Mécanismes de sécurité :

- Validation côté serveur avec Bean Validation
- Protection CSRF intégrée par Spring Security
- Échappement HTML automatique avec Thymeleaf
- Gestion des exceptions avec messages utilisateur

6.3 Optimisation des Performances

Plusieurs techniques d'optimisation ont été implémentées :

6.3.1 Lazy Loading

Listing 13 – Configuration du lazy loading

```
1 @Entity
2 public class Filiere {
3
4     @OneToMany(mappedBy = "filiere", fetch = FetchType.LAZY)
5     private List<Eleve> eleves;
6
7     @OneToMany(mappedBy = "filiere", fetch = FetchType.LAZY)
8     private List<Cours> cours;
9 }
```

7 DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

Le développement de cette application a présenté plusieurs défis techniques qui ont été résolus grâce à une approche méthodique et à la documentation exhaustive de Spring Boot.

7.1 Problème : Gestion des Relations ManyToMany Bidirectionnelles

Défi : Configuration correcte des relations ManyToMany entre Élèves et Cours sans créer de tables redondantes.

Solution : Utilisation appropriée de @JoinTable et mappedBy :

```

1 // Dans Eleve.java
2 @ManyToMany
3 @ManyToOne
4     @JoinColumn(name = "filiere_id")
5     private Filiere filiere;
6
7     @ManyToMany
8     @JoinTable(
9         name = "eleve_cours",
10        joinColumns = @JoinColumn(name = "eleve_id"),
11        inverseJoinColumns = @JoinColumn(name = "cours_id")
12    )
13    private List<Cours> cours = new ArrayList<>();
14
15    @OneToOne(mappedBy = "eleve", cascade = CascadeType.ALL,
16        orphanRemoval = true)
17    private DossierAdministratif dossierAdministratif;
18
19    public void setDossierAdministratif(DossierAdministratif d) {
20        this.dossierAdministratif = d;
21        if (d != null) d.setEleve(this);
22    }

```

7.2 Problème : Génération Automatique du Dossier Administratif

Défi : Création simultanée de l'élève et de son dossier avec dépendance circulaire.

Solution : Utilisation de cascades et séquence de sauvegarde contrôlée :

```

1 @OneToOne(mappedBy = "eleve", cascade = CascadeType.ALL, orphanRemoval =
2     true)
3 private DossierAdministratif dossierAdministratif;
4
5 // Dans le service
6 eleve = eleveRepository.save(eleve); // Premi re sauvegarde pour
7     obtenir ID
8 dossier.setEleve(eleve);
9 eleve.setDossierAdministratif(dossier);
10 return eleveRepository.save(eleve); // Sauvegarde finale

```

7.3 Problème : Intégration Thymeleaf avec Bootstrap

Défi : Création d'interfaces dynamiques et responsives avec intégration Spring.

Solution : Combinaison de fragments Thymeleaf et de composants Bootstrap :

```

1 <!-- Fragment de navigation -->
2 <nav th:fragment="navbar" class="navbar navbar-expand-lg navbar-dark bg-
3     primary">

```

```

3      <div class="container-fluid">
4          <a class="navbar-brand" th:href="@{/}">Gestion Scolaire</a>
5          <div class="collapse navbar-collapse">
6              <ul class="navbar-nav me-auto">
7                  <li class="nav-item">
8                      <a class="nav-link" th:href="@{/eleves}"
9                          th:classappend="{#HttpServletRequest.requestURI.
10                              contains('eleves')} ? 'active'">
11                          lves
12                      </a>
13                  </li>
14              </ul>
15          </div>
16      </div>
</nav>

```

8 CONCLUSION

Ce projet a permis de démontrer une maîtrise complète du framework **Spring Boot** et des technologies associées dans le développement d'applications web professionnelles. Le système de gestion scolaire développé est non seulement fonctionnel mais aussi robuste, maintenable et évolutif, répondant aux standards actuels du développement logiciel.

Les principaux acquis techniques incluent :

- **Architecture Spring Boot** : Maîtrise de l'auto-configuration, des starters et de l'écosystème Spring
- **Persistance avec JPA** : Conception de modèles de données complexes avec relations avancées
- **Développement web** : Création d'interfaces utilisateur modernes avec Thymeleaf et Bootstrap
- **Gestion des transactions** : Implémentation de mécanismes transactionnels robustes
- **Bonnes pratiques** : Application des principes SOLID, DRY et des patterns de conception

Le code produit est **propre, bien documenté et structuré**, suivant les conventions de Spring Boot et les meilleures pratiques de l'industrie. L'application est prête pour un déploiement en environnement de production, avec une architecture qui permet aisément l'ajout de nouvelles fonctionnalités comme :

- Intégration de Spring Security pour l'authentification
- Migration vers une base de données production (PostgreSQL, MySQL)
- Développement d'une API REST complète
- Mise en place de systèmes de reporting avancés
- Intégration avec d'autres systèmes (messagerie, calendrier)

Ce projet constitue une base solide pour des développements futurs plus complexes et démontre les compétences techniques nécessaires pour participer à des projets Spring Boot de grande envergure.